Serenity Research Labs: no purchase event exists on this site

Prepared 7 September 2026 for the Serenity Research Labs development team. Business `biz_63dTkFj3dMwaJN` / `serenityresearchlabs.com`.

Short version. The Whop pixel is installed and working on this site. What is missing is the purchase event specifically. The tracker loads, page views and checkout starts are being recorded, but **nothing at all is sent when an order completes**, so a sale on this store could never appear in Whop even if one came in today.

1. What was measured, and how

Every figure in this document was read live from the Whop events API on 7 September 2026, over a 28 August 2026 to 7 September 2026 (ten days) window. The sweep followed the API's paging cursors to the end and returned **304 events across 1 page**. A partial read was not accepted: the tool used refuses to write a result at all if any page fails, because presence survives a truncated list but **absence does not**.

> **One reading trap worth naming, because it silently produces a wrong answer.** On `GET /events` every custom event, purchases included, comes back with `event_name: "pixel.custom"`. The real name lives in a separate `custom_name` field. Filtering on `event_name` for "purchase" returns zero even on an account that provably has purchases. Every count below was taken across both fields.

2. What is installed, and what is not

what	state
Whop tracker script (`t.whop.tw`)	present on the site
`whop.track` function	present
Page view events	145 recorded
Checkout start events	6 recorded, as `initiate_checkout`
Add to cart events	1
Purchase events	**none, ever**
`whop_purchase` anywhere in the site source	**not present**

> **This is not simply "the store has had no sales".** Those two situations look identical in the data, so they were separated directly: the site source was checked for a purchase sender and there is none. The tracker and the loader are both there, and `whop_purchase` appears nowhere. So the absence of purchase events is a missing install, not an absence of orders. A 404 control was run against the same host first, so the pages that returned content really did return content.

The six `initiate_checkout` events confirm the rest of the pixel is wired up correctly and reaching Whop. Only the final step is missing.

3. Where the event has to come from

Checkout on this store runs through builder generated pages on your own domain (`/checkout/code.html`), served through Cloudflare. That means there is no WordPress or hosted store backend to drop a plugin into, and the purchase call has to be made from whatever code confirms that a payment succeeded.

> **One thing this document cannot tell you, and will not guess at.** The measurement above establishes what reaches Whop and what the site serves. It does not establish where your payments are actually confirmed, and that is the one detail that decides where the call below belongs. If payment is confirmed by a third party checkout, the call should be made from its webhook handler. If you have your own backend, it belongs at the point the order is marked paid. Please place it accordingly rather than wiring it to a page load.

It is worth doing this server side rather than adding a browser event to the confirmation page. A browser fired purchase reports nothing when the customer closes the tab, when a blocker stops the request, or when payment confirms asynchronously. Since this event is being built from scratch, there is no reason to build in that weakness.

4. What to build

One server side call, made from wherever your code already knows an order is paid for. The contract is:

```
POST https://api.whop.com/api/v1/events
Authorization: Bearer <your Whop API key>
Content-Type: application/json

{
  "event_name": "whop_purchase",
  "company_id": "biz_63dTkFj3dMwaJN",
  "event_id":   "<stable unique id for this order>",
  "value":      129.99,
  "currency":   "usd",
  "url":        "https://serenityresearchlabs.com/...",
  "user":    { "anonymous_id": "<_wuid cookie>", "email": "buyer@example.com" },
  "context": { "user_agent": "<the buyer's user agent>" }
}
```

Note that on the **send** side the event name goes in `event_name`. On the read side the same event comes back as `pixel.custom` with the name in `custom_name`. That asymmetry is Whop's, not a mistake in this document.

The example below is written as a Node handler. The contract above is plain HTTP, so any language works. What matters is that it runs where the payment is confirmed, and that `event_id` comes from the order.

4a. The attribution join, which is the part that is easy to miss

A server side event with no `user.anonymous_id` is a sale Whop cannot connect to the ad that produced it. It will be counted as revenue and attributed to nothing. The value comes from the `_wuid` cookie that the Whop pixel already sets in the buyer's browser, so it has to be captured in the browser and carried through to the server.

```
/* Runs in the browser at checkout, BEFORE the order is submitted. */
/* _wuid is the visitor id the Whop pixel already sets on your site. */
/* Without it the server event cannot be joined to the ad click, and */
/* the sale reports as unattributed. */
function whopWuid() {
  const m = document.cookie.match(/(?:^|;\s*)_wuid=([^;]+)/);
  return m ? decodeURIComponent(m[1]) : null;
}

/* Persist it ON THE ORDER RECORD, alongside the order id, so the */
/* server still has it when the payment confirms minutes later. */
orderPayload.whop_wuid = whopWuid();
```

Capture it at checkout, not at confirmation. If the payment confirms after the customer has closed the tab, there is no browser left to read the cookie from. The whole point of moving this server side is that the tab is no longer required, and reading the cookie late gives that dependency straight back.

4b. The endpoint

```
/* app/api/whop-purchase/route.js  Next.js App Router on Vercel. */
/* Call this once from the code path that CONFIRMS an order, server side. */
/* Never from the browser: the customer's tab is not involved. */

/* Server env var only. Never NEXT_PUBLIC_. */
const WHOP_KEY = process.env.WHOP_API_KEY;
const COMPANY  = 'biz_63dTkFj3dMwaJN';

export async function POST(req) {
  const order = await req.json();

  /* event_id is what makes this safe to call more than once. Whop */
  /* de-duplicates on it, so a retry, a refresh or a replayed webhook */
  /* cannot double count the same order. */
  const body = {
    event_name: 'whop_purchase',
    company_id: COMPANY,
    event_id:   'purchase_' + order.id,
    value:      Number(order.total),
    currency:   'usd',
    url:        'https://serenityresearchlabs.com/checkout/code.html',
    user: {
      anonymous_id: order.whop_wuid || undefined,
      email:        order.email     || undefined
    },
    context: { user_agent: order.user_agent || undefined }
  };

  const r = await fetch('https://api.whop.com/api/v1/events', {
    method:  'POST',
    headers: {
      'Authorization': 'Bearer ' + WHOP_KEY,
      'Content-Type':  'application/json'
    },
    body: JSON.stringify(body)
  });

  if (!r.ok) {
    /* Log it and retry out of band. Do NOT fail the customer's order */
    /* because a tracking call failed. */
    console.error('whop purchase post failed', r.status, await r.text());
  }
  return new Response(null, { status: 204 });
}
```

Keep the API key server side. On Vercel that means a plain environment variable, not one prefixed `NEXT_PUBLIC_`. A key exposed to the browser can be used by anyone to write events into your account.

5. How to confirm it works

Place one real order, then read the account back:

```
curl -s -H "Authorization: Bearer $WHOP_API_KEY" \\
  "https://api.whop.com/api/v1/events?company_id=biz_63dTkFj3dMwaJN&from=2026-09-
07T00:00:00Z&to=2026-09-08T00:00:00Z"
```

You are looking for a row with `custom_name: "whop_purchase"` carrying the order's real total in `total_usd_amount`. Two further checks separate a real server side install from a browser one:

1. **Send it twice.** Post the same `event_id` again. The event count must not go up. If it does, the idempotency key is not being set from anything stable.
2. **Kill the tab.** Complete a checkout and close the browser before the confirmation page renders. The purchase event must still arrive. This is the entire reason for the work, and it is the one test a browser install cannot pass.

> Whop's event stream lags ingestion by a few minutes, and an empty read taken too soon is not evidence of a failure. If a window looks empty, widen it to several days before concluding anything. A single empty response has produced more than one false alarm on this platform.